CNN Weight Vault 發展日誌 - 0504

目錄

1. 今日重點
2. 泛化權重管理系統
3. 實作改動
4. 使用方法
5. 測試結果
6. 技術細節
7. 流程圖

今日重點

目標

實現通用的權重存取方式，讓任意 PyTorch 模型都能使用向量資料庫。

問題

原本的 `DBConv2d` / `DBLinear` 類需要繼承修改模型定義，不夠泛化。

解決方案

- 訓練後：從任意模型提取 `state_dict` ，存儲到 vault
- 初始化時：從 vault 檢索權重，載入到相同架構的模型

泛化權重管理系統

核心概念

```
任意 PyTorch 模型 → extract_weights() → Qdrant Cloud
                                        ↓
任意 PyTorch 模型 ← load_weights() ← Qdrant Cloud
```

關鍵特性

特性	說明
層類型無關	不管裡面是 Conv2d、Linear、BatchNorm、Embedding 都能處理

特性	說明
模型無關	支持 ResNet、VGG、MobileNet、自定義模型
無需修改模型	直接操作 `state_dict` ，不需繼承特定類別
自動指紋壓縮	大維度層（>65535）自動使用指紋壓縮

實作改動

1. 新增 `cnn_weight_vault/wrap.py`

提供通用權重管理函數：

- `extract_weights()` - 存儲權重
- `load_weights()` - 載入權重
- `save_model_to_vault()` / `load_model_from_vault()` - 整體模型操作
- `TrainingHook` - 訓練鉤子，自動存儲

2. 修改 `qdrant_vault.py`

新增方法：

- `_store_raw(vector, layer_key, metadata)` - 通用存儲（支持指紋壓縮）
- `_get_weights_by_key(layer_key, shape, object_category, force)` - 通用檢索

修復問題：

- 指紋壓縮：大維度層自動壓縮到 32768 維
- API 兼容性：修復 `config.vectors` vs `vectors_config` 問題
- 向量轉換：確保 list 轉為 numpy array

3. 更新 `__init__.py`

導出新模組：

```
from .wrap import (
    extract_weights,
    load_weights,
    save_model_to_vault,
    load_model_from_vault,
    find_similar_model,
    TrainingHook
)
```

使用方法

基本用法

```
from cnn_weight_vault import QdrantWeightVault, extract_weights, load_weights

# 1. 創建 vault
vault = QdrantWeightVault(collection_name="my_vault")

# 2. 訓練後存儲權重
extract_weights(
    model=trained_model,
    vault=vault,
    model_name="resnet18",
    epoch=10,
    accuracy=95.5,
    object_category="cat"
)

# 3. 初始化時載入權重
new_model = models.resnet18(weights=None)
results = load_weights(
    model=new_model,
    vault=vault,
    object_category="cat",
    force=True  # True: 強制載入, False: 相似度閾值內才載入
)
```

TrainingHook

```
from cnn_weight_vault import TrainingHook

hook = TrainingHook(vault, model_name="resnet18", save_interval=5)
hook.register(model)

for epoch in range(num_epochs):
    train(model)
    hook.on_epoch_end(model, epoch, accuracy)
```

Hook 的優點

優點	說明
自動存儲	不會忘記存權重
間隔存儲	可以設定每 N 個 epoch 存一次
追蹤進度	保存每個 epoch 的權重，方便比較
構建 vault	逐步建立向量資料庫

測試結果

ResNet18 測試

```
[WeightStore] Stored 122 weight tensors to vault (model=resnet18)
[WeightLoader] Loaded 122/122 layers from vault
```

- **存儲**：122 層權重
- **載入**：122/122 層 (100%)
- 指紋壓縮：大維度層（>65535維）自動壓縮到 32768 維

層結構分析

模型	總層數	類型
ResNet18	122	conv, linear, bn, bias
ResNet50	320	conv, linear, bn, bias
VGG16	32	conv, linear
MobileNetV2	314	conv, linear, bn

技術細節

指紋壓縮 (Fingerprint)

當權重向量維度超過 Qdrant 免費版限制（65535）時：

- 使用 Count Sketch 演算法壓縮到 32768 維
- 存儲時標記 `use_fingerprint=True`
- 檢索時重建近似值

Layer Key

根據形狀生成唯一鍵值：

權重形狀	Layer Key
(64, 3, 7, 7)	`conv_64_3_7_7`
(512, 512)	`linear_512_512`
(64,)	`bias_64` 或 `1d_64`

相似度檢索模式

模式	說明
`force=True`	直接獲取最新存儲的權重
`force=False`	使用餘弦相似度，僅在相似度 > 閾值時載入

關鍵設計

設計	說明
layer_key	根據權重形狀生成，讓同類型的層共享 vault 記錄
top-k 遮罩	只保留 30% 最重要的權重，減少雜訊
指紋壓縮	大維度層 (>65535) 壓縮到 32768 維，適配 Qdrant 限制
force 模式	True = 強制載入，False = 相似度閾值檢查

流程圖

儲存流程 (Training)

```
model.state_dict()
       ↓
for each layer:
       ↓
_generate_layer_key()  → "conv_64_3_7_7"
       ↓
_apply_top_k_mask()    → 保留 top 30% 權重
       ↓
vault._store_raw()     → 存入 Qdrant Cloud
```

載入流程 (Init)

```
new_model.state_dict()
       ↓
for each layer:
       ↓
_generate_layer_key()           → "conv_64_3_7_7"
       ↓
vault._get_weights_by_key()     → 查詢 vault
       ↓
if force: 直接取最新
else:     相似度 > 閾值才取
       ↓
model.state_dict()[layer].copy_(weights)
```

詳細實作

extract_weights() 實現

```
def extract_weights(model, vault, model_name, epoch, accuracy, object_category):
    # Step 1: 取得模型的所有權重
    state_dict = model.state_dict()

    for layer_name, weight_tensor in state_dict.items():
        # Step 2: 根據形狀生成 layer_key
```

```
        layer_key = _generate_layer_key(weight_tensor.shape, layer_name)

        # Step 3: 應用 top-k 遮罩（保留最重要的 30% 權重）
        masked_weights, mask = _apply_top_k_mask(weight_tensor, top_k_ratio=0.3)

        # Step 4: 展平為向量
        flattened = masked_weights.flatten().numpy().astype(np.float32)

        # Step 5: 存入 vault
        vault._store_raw(
            vector=flattened,
            layer_key=layer_key,
            metadata={
                'layer_name': layer_name,
                'model_name': model_name,
                'epoch': epoch,
                'accuracy': accuracy,
                'shape': list(weight_tensor.shape),
                'object_category': object_category
            }
        )
```

load_weights() 實現

```
def load_weights(model, vault, object_category, force=False):
    state_dict = model.state_dict()
    results = {}

    for layer_name, weight_tensor in state_dict.items():
        layer_key = _generate_layer_key(weight_tensor.shape, layer_name)

        weights = vault._get_weights_by_key(
            layer_key=layer_key,
            shape=weight_tensor.shape,
            object_category=object_category,
            force=force
        )

        if weights is not None and weights.shape == weight_tensor.shape:
            with torch.no_grad():
                model.state_dict()[layer_name].copy_(weights)
            results[layer_name] = True
        else:
            results[layer_name] = False

    return results
```

Vault 內部 (_store_raw)

```
def _store_raw(self, vector, layer_key, metadata):
    # 1. 檢查維度是否超過 Qdrant 限制 (65535)
    if len(vector) > 65535:
        # 2. 指紋壓縮：大維度 → 32768 維
        tensor = torch.from_numpy(vector).reshape(tuple(metadata['shape']))
        vector = self._compute_fingerprint(tensor, target_dim=32768)
        metadata['use_fingerprint'] = True

    # 3. 創建 collection（如果不存在）
    collection_name = self._get_collection_name(layer_key)
    self._ensure_collection_exists(collection_name, dimension=len(vector))

    # 4. 存入 Qdrant Cloud
    self.client.upsert(
        collection_name=collection_name,
        points=[PointStruct(
            id=uuid4(),
            vector=vector.tolist(),
            payload=metadata
        )]
    )
```

更新日期：2025-05-03